

SIMATS ENGINEERING

ASSESSMENT TOOL 3 — REVERSE ENGINEERING TASK

Course Name: Object Oriented Analysis and Design

Course Code: CSA1105

Course Outcome: CO4 — Implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Weightage: 30%

Questions Answered: 3 of 6 — Hotel Reservation, Transportation Management, Healthcare Monitoring | **Total Marks:** 30

Code of Conduct:

I __Rashmi Naurene9 192521357)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Rashmi*

Q1. Legacy Hotel Reservation System [10 Marks]

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules. Analyze the existing architecture and identify problems related to coupling, duplication, and persistence management. Reverse engineer the system by introducing suitable DAO, Repository, or Data Access patterns, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

1. UNDERSTANDING OF THE SYSTEM

The reservation application's screen classes — for rooms, guests, bookings, and payments — each embed their own OODBMS query calls directly inside UI event handlers. There is no separate layer responsible for persistence; whichever screen needs data simply talks to the database itself, and several screens independently re-implement near-identical queries such as 'find guest by ID' or 'find available rooms for a date range'.

2. DECOMPOSITION — PROBLEMS IN THE EXISTING DESIGN

- **Tight coupling to the database** — screen classes (RoomScreen, GuestScreen, BookingScreen, PaymentScreen) are directly coupled to the OODBMS API, so any change to the database technology or query structure forces changes across every screen that touches that data.
- **Duplicated query logic** — the same lookups (find room by number, find guest by ID, find bookings by date) are copy-pasted into multiple screens instead of being written once and reused.
- **No separation of concerns** — UI classes are simultaneously responsible for presentation, input validation, and persistence, which violates single responsibility and makes each class harder to read, test, and modify safely.
- **Poor persistence management** — there is no single place that governs how objects are created, updated, or retrieved from the OODBMS, making it difficult to introduce caching, transactions, or connection pooling consistently.

3. IDENTIFICATION OF COMPONENTS — CLASSES, ATTRIBUTES, OPERATIONS

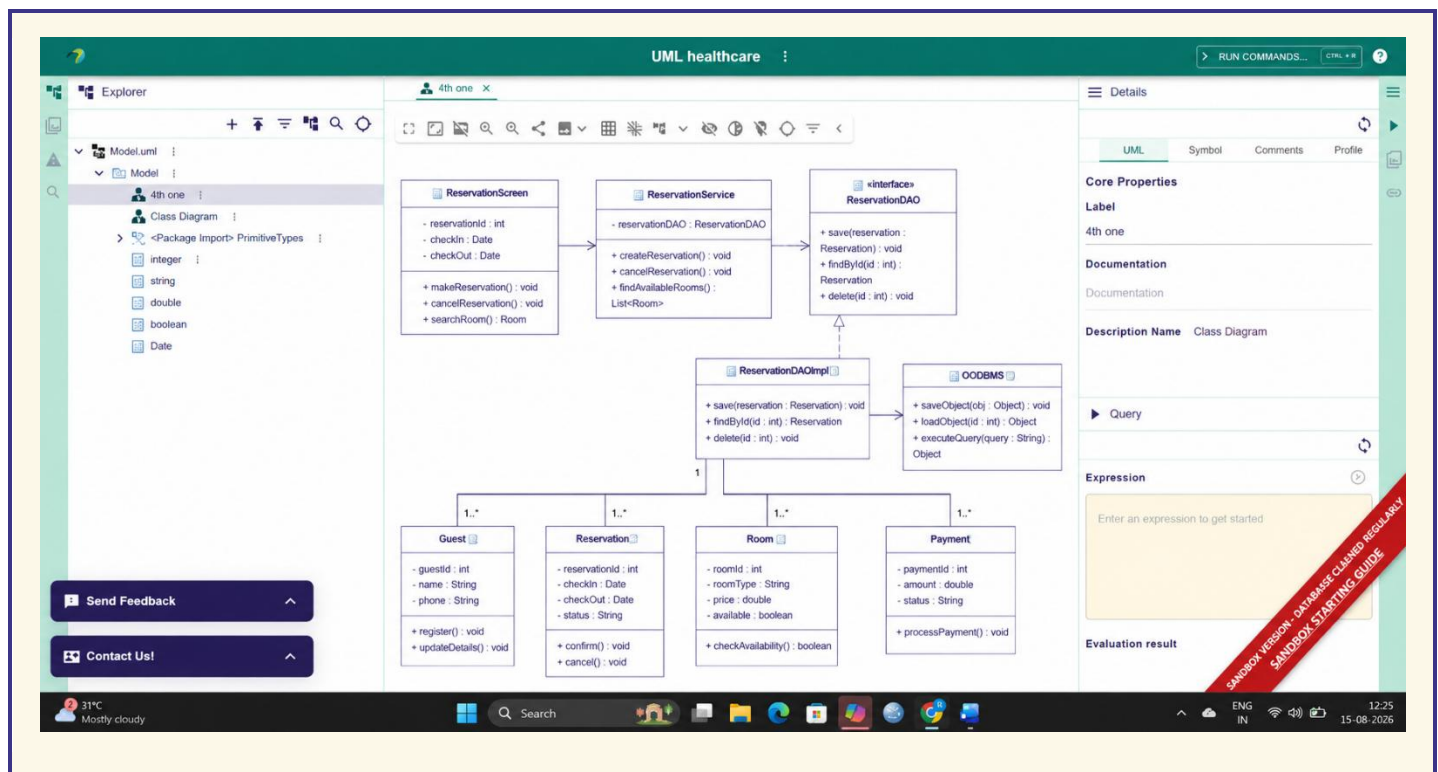
Class	Key Attributes	Key Operations
Room	roomNo, type, rate	checkAvailability()
Guest	guestId, name, contact	updateDetails()
Booking	bookingId, checkIn, checkOut	confirmBooking(), cancelBooking()
Payment	paymentId, amount, status	processPayment()
RoomDAO / GuestDAO / BookingDAO / PaymentDAO (interfaces)	—	findById(), save(), update(), delete()
RoomDAOImpl / GuestDAOImpl /	— (holds the OODBMS connection)	implements the DAO interface's operations

Class	Key Attributes	Key Operations
BookingDAOImpl / PaymentDAOImpl		
ReservationService	—	bookRoom(), cancelReservation()
ReservationScreen	—	displayAvailableRooms(), submitBooking()

Relationships: ReservationScreen depends on ReservationService only, never on a DAO or the database directly. ReservationService depends on the four DAO interfaces (RoomDAO, GuestDAO, BookingDAO, PaymentDAO), not their concrete implementations. Each concrete *Impl class realises its corresponding interface. Booking associates with Room (1 to 1), Guest (1 to 1), and is composed with Payment (1 to 1, a payment record has no meaning outside its booking).

4. RECONSTRUCTION — REDESIGNED ARCHITECTURE

The system is restructured into four layers: a Presentation layer (screens), a Service layer (business logic such as ReservationService), a Data Access layer (the DAO interfaces and their concrete implementations), and the OODBMS itself. Every screen now calls the service layer only; every service calls a DAO interface only, never a concrete class or the database directly. This is the standard DAO pattern — it can optionally be extended with a Repository class (e.g. BookingRepository) that composes several DAOs together when a single business operation needs data from more than one entity at once.



5. IMPACT OF THE REDESIGN

- **Modularity** — each DAO is responsible for exactly one entity's persistence, so a change to how bookings are stored no longer risks breaking room or payment queries.

- **Reusability** — the same DAO is now called from every screen and service that needs that entity, eliminating the duplicated query logic identified above.
- **OODBMS integration** — all database-specific code is isolated inside the *Impl classes, so the OODBMS technology can be upgraded or swapped by changing only those classes, with the service and presentation layers completely unaffected.

Q2. Legacy Transportation Management System [10 Marks]

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services. Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable creational design patterns, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

1. UNDERSTANDING OF THE SYSTEM

Bus, Train, Taxi, and RentalVehicle are handled as four separate, largely independent modules. Wherever the application needs a transport object — in booking, scheduling, or pricing — it directly instantiates the concrete class it needs (for example, calling 'new Bus()' or 'new Train()' inline). Because this instantiation logic is scattered across many call sites rather than centralised, adding a new transport type such as a Ferry means hunting down and editing every one of those call sites.

2. DECOMPOSITION — PROBLEMS IN THE EXISTING DESIGN

- **Duplicated creation logic** — the same 'new Bus()' / 'new Train()' / 'new Taxi()' style instantiation is repeated in the booking, scheduling, and pricing modules instead of being written once.
- **No common abstraction** — Bus, Train, Taxi, and RentalVehicle are not related through a shared interface or base class, even though they all represent the same underlying concept — a bookable transport option — so client code cannot treat them uniformly.
- **Poor extensibility** — introducing a new transport type requires modifying several existing modules rather than simply adding one new class, which directly violates the Open/Closed Principle.

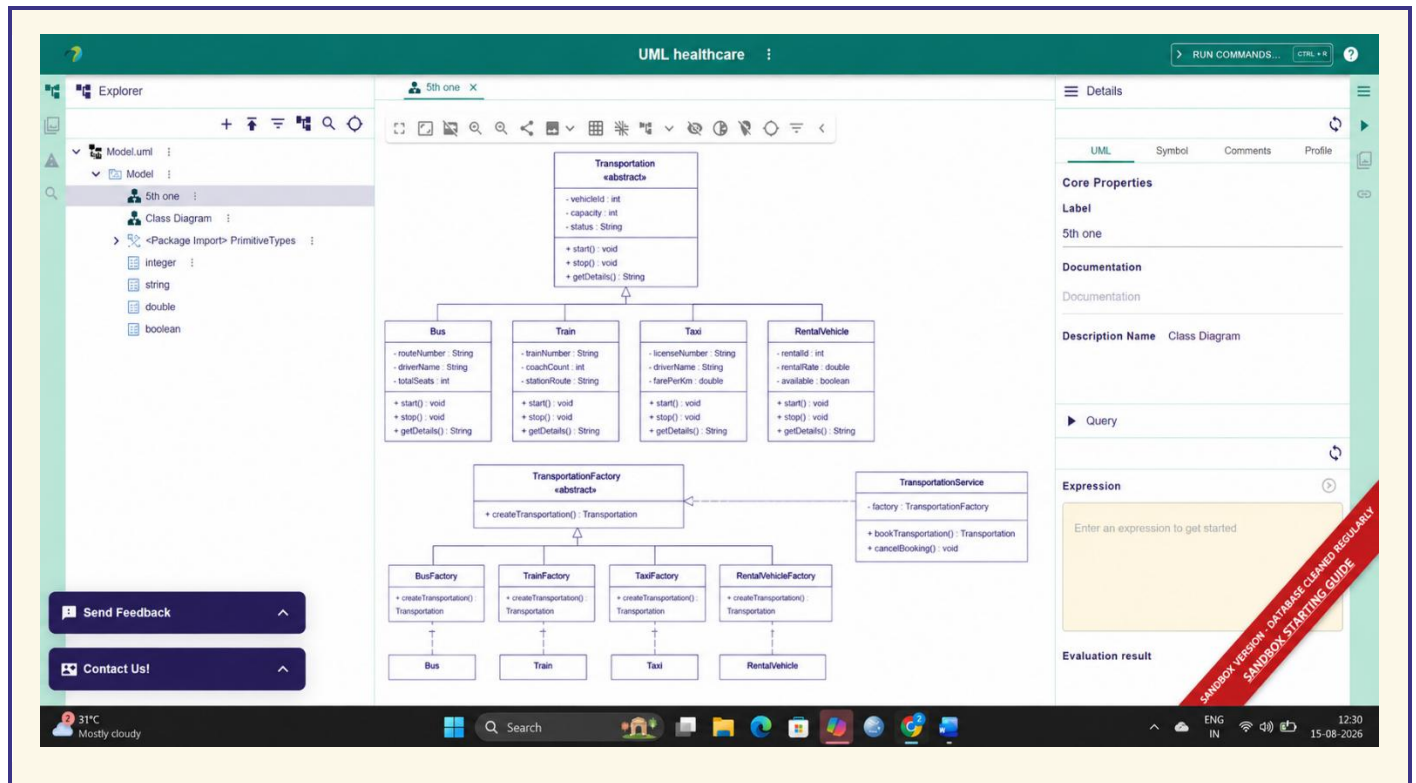
3. IDENTIFICATION OF COMPONENTS — CLASSES, ATTRIBUTES, OPERATIONS

Class	Key Attributes	Key Operations
Vehicle (abstract)	vehicleId, capacity, route	calculateFare(), getSchedule()
Bus / Train / Taxi / RentalVehicle	type-specific fields (e.g. seatClass for Train)	calculateFare() — overridden per type
TransportFactory (abstract)	—	createTransport() : Vehicle
BusFactory / TrainFactory / TaxiFactory / RentalVehicleFactory	—	createTransport() — overridden, returns the matching Vehicle subtype
BookingService	—	bookTransport(factory)

Relationships: Bus, Train, Taxi, and RentalVehicle generalise from the abstract Vehicle class. BusFactory, TrainFactory, TaxiFactory, and RentalVehicleFactory generalise from the abstract TransportFactory class, and each concrete factory's createTransport() depends on (creates) its matching Vehicle subclass. BookingService depends only on the abstract TransportFactory type, never on a concrete factory or a concrete Vehicle subclass directly.

4. RECONSTRUCTION — REDESIGNED ARCHITECTURE

The Factory Method pattern is applied here: an abstract `TransportFactory` class declares `createTransport()`, and each concrete factory (`BusFactory`, `TrainFactory`, and so on) overrides it to instantiate its own `Vehicle` subtype. `BookingService` and every other client module now ask a factory for a `Vehicle` instead of writing 'new Bus()' themselves, so the decision of which concrete class to instantiate is centralised in exactly one place per transport type. Because `Vehicle` itself is a common abstraction, this also enables polymorphism: `BookingService` can call `vehicle.calculateFare()` on any transport object without knowing or checking its concrete type.



5. IMPACT OF THE REDESIGN

- **Extensibility** — a new transport type, such as Ferry, is added by writing one new `Vehicle` subclass and one new factory subclass; no existing class needs to be modified, satisfying the Open/Closed Principle.
- **Scalability** — as the number of transport types grows, creation logic stays centralised and manageable instead of sprawling further across the codebase.
- **Code reuse** — common `Vehicle` behaviour is written once and inherited by every subtype, and every client module reuses the same factory interface instead of duplicating instantiation code.

Q3. Legacy Healthcare Monitoring System [10 Marks]

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components. Reverse engineer the application by identifying violations of SOLID principles, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

1. UNDERSTANDING OF THE SYSTEM

A small number of classes — most notably a single PatientMonitor-style class — currently handle the entire pipeline: reading sensor values, deciding whether an alert is needed, saving records, and producing reports. This class also creates and calls specific sensor and storage classes directly (for example a concrete HeartRateSensorDevice and a concrete SQLStorage class) rather than working through any abstraction.

2. DECOMPOSITION — SOLID VIOLATIONS IDENTIFIED

- **SRP violation** — one class is responsible for sensor reading, alert generation, record storage, and report generation at once — four unrelated reasons for that class to change, which is the textbook definition of a Single Responsibility Principle violation.
- **OCP violation** — adding a new sensor type or a new storage backend means editing the existing class's internal logic (often through added conditional branches) instead of extending the system by adding a new class, which violates the Open/Closed Principle.
- **DIP violation** — the high-level monitoring logic depends directly on concrete device and storage classes rather than on abstractions, so the Dependency Inversion Principle is violated — high-level modules should depend on interfaces, not on low-level implementation details.

3. IDENTIFICATION OF COMPONENTS — CLASSES, ATTRIBUTES, OPERATIONS

Class	Key Attributes	Key Operations
SensorReader (interface)	—	readValue()
HeartRateSensor / TemperatureSensor / OxygenSensor	sensorId	readValue() — implemented per device
AlertGenerator	thresholds	generateAlert(reading) : Alert
Alert	alertId, severity, message	—
MedicalRecordRepository (interface)	—	save(record), findByPatientId(id)
ConcreteRecordRepository	— (holds the OODBMS connection)	implements the repository interface
ReportGenerator	—	generateReport(patientId) : Report
PatientMonitorService	—	monitorPatient()

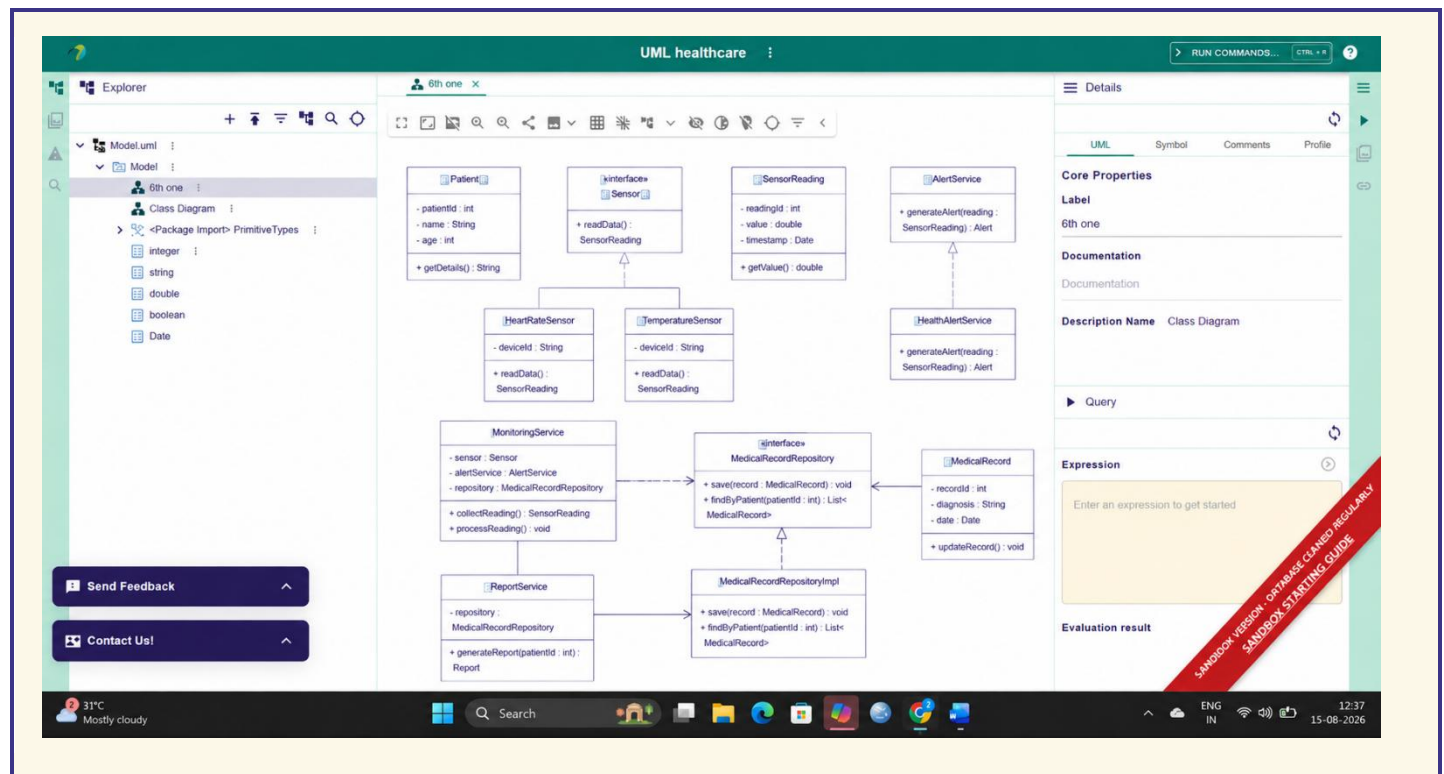
Relationships: HeartRateSensor, TemperatureSensor, and OxygenSensor each realise the SensorReader interface. ConcreteRecordRepository realises the MedicalRecordRepository interface.

PatientMonitorService depends only on the SensorReader and MedicalRecordRepository interfaces (never on a concrete sensor or storage class) and associates with AlertGenerator and ReportGenerator to delegate alert and report creation respectively.

4. RECONSTRUCTION — REDESIGNED ARCHITECTURE

Each responsibility is pulled out into its own cohesive class: SensorReader becomes an interface implemented separately by each device type; AlertGenerator only evaluates readings and produces Alert objects; MedicalRecordRepository becomes an interface implemented by whatever concrete storage mechanism is in use; and ReportGenerator only produces reports from stored records.

PatientMonitorService is left as a thin coordinator that calls these collaborators through their interfaces, never instantiating or depending on a concrete device or storage class directly. This single change — depending on interfaces instead of concrete classes — is what restores the Dependency Inversion Principle, and splitting the one large class into these five focused classes is what restores the Single Responsibility Principle.



5. IMPACT OF THE REDESIGN

- **Cohesion** — every class now has exactly one reason to change, so `SensorReader` implementations, `AlertGenerator`, and `ReportGenerator` can each be understood and modified in isolation.
- **Loose coupling** — `PatientMonitorService` no longer depends on any concrete device or storage class, only on interfaces, so those implementations can change freely without touching the coordinating class.

- **Flexibility** — a new sensor type or a new storage backend is added by writing one new class that implements the existing interface, with zero changes to PatientMonitorService, restoring the Open/Closed Principle.
- **Testability** — PatientMonitorService can now be unit-tested using mock SensorReader and MedicalRecordRepository implementations, without needing real hardware or a live database connection.
- **Maintainability** — smaller, focused classes are easier to locate, read, and safely fix compared to one large class handling four unrelated concerns.